A Course Based Project on

Design and Implementation of a Hardware Watchdog for a RISC-V Processor

Submitted to the Department of ECE

in partial fulfillment of the requirements for the completion of course

FIELD PROJECT (22SD5EC202)

BACHELOR OF TECHNOLOGY
in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Siddamshetti Manaswini	24071A04J7
Yerra Venkata Mokshagna	24071A04K8

UNDER THE SUPERVISION OF

Mr. R. Ravi Kumar, Assistant Professor

TAMASOMA JYOTIRGAMAYA
VIGNANA JYOTHI

VNRVJIET
VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
AUTONOMOUS INSTITUTE

A.Y. 2025-26

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the course based project entitled "Design and Implementation of a Hardware Watchdog for a RISC-V Processor" is being submitted by Siddamshetti Manaswini (24071A04J7), Yerra Venkata Mokshagna(24071A04K8), for the partial fulfillment of requirements for course based project of **Field Project** in II year II semester of Bachelor of Technology in Electronics and Communication Engineering of VNRVJIET, Hyderabad during the academic year 2025 - 2026.

Course Coordinator
Mr. R. Ravi Kumar
Assistant Professor
VNRVJIET, Hyd.

Head of the Department
Dr. P. Kishore
Assoc.Professor & Head

DECLARATION

We do declare that the course based project report entitled "Design and Implementation of a Hardware Watchdog for a RISC-V Processor" submitted to the Department of Electronics and Communication Engineering, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfillment of the requirements for course based project of Field Project in II B.Tech. II Semester for the academic year 2025 - 2026.

Place: Hyderabad Date:

	Student Name	Roll No.	Student Signature
1.	Siddamshetti Manaswini	24071A04J7	
2.	Yerra Venkata Mokshagna	24071A04K8	

Verified by:

Mr. R. Ravi Kumar,
Assistant Professor

Date of Verification:

CONTENTS

PAGE NO:

ABSTRACT

This project presents the design and verification of a simplified RISC-V instruction fetch model integrated with a hardware watchdog timer for fault detection and automatic recovery. The system focuses on the Program Counter (PC) operation and its dependency on memory handshake signals and control conditions. The PC is designed to increment only when valid instruction fetch conditions are satisfied, namely when memory is ready, no stall is asserted, and the system is operating normally.

A dedicated watchdog module continuously monitors the system for four critical fault conditions: trap signals, memory handshake failure, stall conditions, and Program Counter freeze. The watchdog employs a priority-based fault detection mechanism and generates an active-low reset signal when any fault persists beyond a predefined timeout or immediately in the case of critical faults such as traps. A one-shot reset mechanism ensures that the system is not repeatedly reset for the same fault event.

A structured testbench is developed to simulate and validate system behavior across multiple scenarios. Each test case follows a controlled sequence of normal operation, fault injection, watchdog reset activation, system recovery, and gap intervals. This approach ensures clear observation of system behavior, including PC progression, fault detection, reset generation, and recovery.

The results demonstrate that the watchdog successfully identifies faults and restores normal operation by resetting the core, thereby improving system reliability. The project provides a clear understanding of fault-tolerant design principles and lays a foundation for extending the model to full RISC-V processors and real-time embedded systems.

1. INTRODUCTION

1.1 motivation

Modern processor systems are expected to operate continuously and reliably, but simple designs often lack mechanisms to handle faults such as memory failures, execution stalls, or program counter freezes. These issues can cause the system to hang indefinitely, making it difficult to diagnose and recover without external intervention. This project is motivated by the need to introduce hardware-based fault detection and automatic recovery using a watchdog timer that continuously monitors system behavior. By integrating the watchdog with a simplified RISC-V core, the system can detect abnormal conditions and reset itself, ensuring reliable operation. The project also aims to provide a clear understanding of Program Counter (PC) behavior under different conditions and demonstrate how real-world systems maintain stability. Overall, it bridges theoretical concepts with practical implementation, helping in the design of more robust and fault-tolerant digital systems.

1.2 Objectives

The main objectives of this project are:

- To design a simplified RISC-V instruction fetch model focusing on Program Counter (PC) behavior
- To implement a hardware watchdog timer for fault detection and system monitoring
- To detect key fault conditions such as trap, memory failure, stall, and PC freeze
- To generate an automatic reset signal when faults persist or critical errors occur
- To ensure system recovery after reset and resume normal operation
- To develop a structured testbench with clear test cases for observing system behavior
- To analyze and demonstrate fault-tolerant system design in a clear and explainable manner

1.3 Scope

This project focuses on the design, simulation, and analysis of a simplified RISC-V instruction fetch model integrated with a hardware watchdog timer for fault detection and automatic recovery. The scope includes modeling the behavior of the Program Counter (PC) based on memory handshake signals and control inputs such as stall and trap conditions. It also involves the design of a watchdog module that continuously monitors the system for critical faults, including memory handshake failure, execution stalls, trap conditions, and PC freeze scenarios, and generates a reset signal when necessary.

A structured and well-defined testbench is developed to simulate multiple fault conditions and observe system behavior through different phases such as normal operation, fault injection, watchdog activation, recovery, and idle gap periods. The project emphasizes clear visualization of system response, making it easier to understand how faults affect execution and how the system recovers.

The scope is limited to the instruction fetch stage of the RISC-V architecture and does not cover full instruction execution, pipelining, cache/memory hierarchy, or physical hardware implementation. However, it establishes a strong conceptual and practical foundation for extending the design to complete processor architectures, real-time embedded systems, FPGA implementations, and advanced fault-tolerant computing applications.

2. LITERATURE SURVEY & REQUIREMENTS

2.1 Existing Systems

In modern digital systems, reliability is typically achieved using a combination of software-based monitoring and hardware watchdog timers. Most embedded processors, microcontrollers, and system-on-chip (SoC) designs include watchdog timers that continuously monitor system activity and trigger a reset if the system becomes unresponsive. These watchdogs are commonly used in applications such as automotive systems, industrial controllers, and consumer electronics to prevent system hangs and ensure continuous operation.

Traditional watchdog systems are often timer-based, where the processor must periodically reset (or "kick") the watchdog to indicate normal operation. If the watchdog is not serviced within a specified time, it assumes a fault and resets the system. While effective, such systems do not always identify the exact cause of the fault and rely heavily on software behavior.

In more advanced designs, hardware fault detection mechanisms are integrated to monitor specific conditions such as memory failures, pipeline stalls, or illegal instructions. These systems may include error detection circuits, exception handling units, and redundancy techniques to improve reliability. However, these implementations are typically complex and part of full-fledged processor architectures.

Compared to existing systems, this project presents a simplified and educational model that combines basic RISC-V instruction fetch behavior with a hardware watchdog capable of detecting multiple fault conditions (trap, memory failure, stall, and PC freeze). This approach provides a clear and structured understanding of fault detection and recovery mechanisms, making it suitable for learning and demonstration purposes while reflecting key concepts used in real-world systems.

2.2 Software Requirements

- **Xilinx Vivado**
  Used for writing, simulating, and analyzing the Verilog design. Vivado provides waveform visualization, debugging tools, and simulation environment to verify the behavior of the RISC-V core and watchdog timer.

3. SYSTEM DESIGN

3.1 System Architecture

The project consists of two major sections:

1. RISC-V Core Unit

This unit models the instruction fetch stage of a processor. It contains the Program Counter (PC), control logic, and memory handshake interface. The PC continuously updates based on system conditions such as memory readiness and stall signals. The core generates mem_valid to request instruction fetch and updates the PC only when valid conditions are satisfied. It also provides outputs like pc and pc_active for monitoring system behavior.

2. Watchdog Timer Unit

This unit continuously monitors the operation of the RISC-V core. It observes signals such as PC, memory handshake (mem_valid, mem_ready), stall, and trap conditions. Based on these signals, it detects faults including memory failure, execution stall, PC freeze, and trap conditions. If any fault persists beyond a set timeout or occurs immediately (in case of trap), the watchdog generates an active-low reset signal (wdt_rst_n) to recover the system. It also produces a fault_code to indicate the type of error detected.

3. Testbench / Control Unit

This unit acts as the external environment controlling the system. It generates input signals such as clock, reset, memory readiness, stall, and trap conditions. It executes structured test cases to simulate normal operation, fault injection, watchdog reset, and system recovery, allowing clear observation of system behavior.

3.2 Flowchart

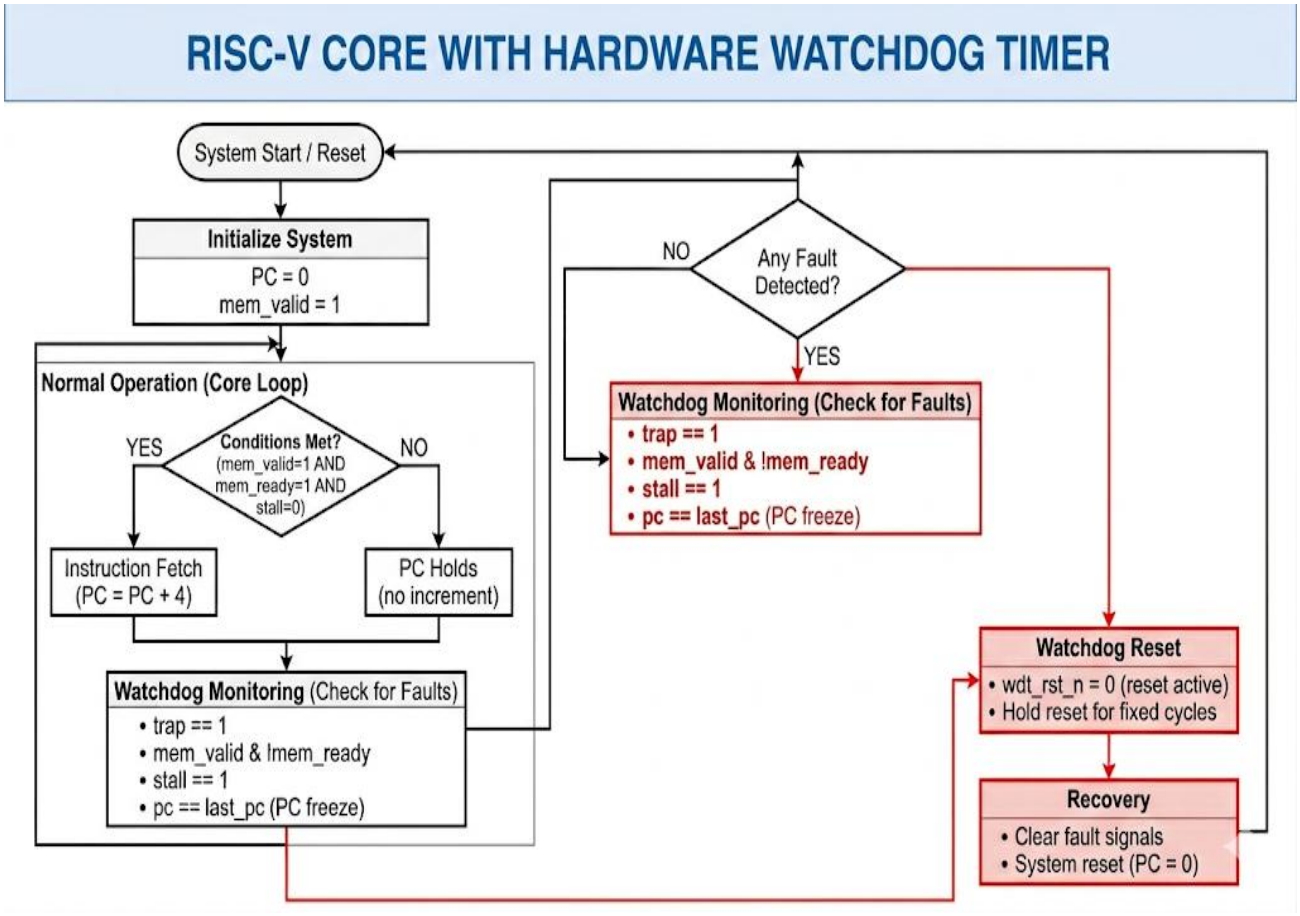

Fig 3.2.1: Flowchart of RISC-V core with Hardware Watchdog

4. IMPLEMENTATION

This chapter explains how the proposed system is implemented, including the overall working process, software logic, and important code segments used in the project

4.1 Basic Process

Initially, the system is powered ON and a global reset (sys_rst_n) is applied to initialize all modules. The RISC-V core starts with the Program Counter (PC) set to zero and begins instruction fetch operation. The core continuously generates a mem_valid signal to request instruction fetch, while the testbench provides the mem_ready signal to simulate memory response.

During normal operation, the PC increments by 4 on every clock cycle when all conditions are satisfied, namely mem_valid = 1, mem_ready = 1, and stall = 0. If any of these conditions fail, the PC holds its value, simulating real processor behavior under stall or memory delay conditions.

The watchdog timer continuously monitors system signals such as PC, memory handshake, stall, and trap conditions. When a fault such as memory failure (mem_ready = 0), stall (stall = 1), trap (trap = 1), or PC freeze (pc == last_pc) is detected, the watchdog starts a timeout counter. If the fault persists beyond a predefined duration, or immediately in the case of a trap, the watchdog generates an active-low reset signal (wdt_rst_n = 0).

Once the reset is triggered, the core is reset, the PC returns to zero, and normal operation resumes after clearing the fault condition. This process demonstrates automatic fault detection and recovery in the system.

4.2 Software Logic

The system is divided into multiple logical modules:

- **RISC-V Core Module:**
  Implements the instruction fetch mechanism. It updates the Program Counter based on memory handshake and control signals. The PC increments only when valid conditions are satisfied; otherwise, it remains unchanged.
- **Watchdog Monitoring Module:**
  Continuously observes system signals to detect faults such as trap, memory failure, stall, and PC freeze. It uses priority logic and timeout counters to determine when to trigger a reset.
- **Fault Detection Module:**
  Identifies specific fault conditions:
  - Trap → immediate reset

- Memory failure → mem_valid & !mem_ready
- Stall → stall = 1
- PC freeze → pc == last_pc

- **Reset Control Module:**

  Combines system reset and watchdog reset to generate core_rst_n. This ensures the processor resets automatically when a fault occurs.

- **Testbench Control Module:**

  Generates input signals such as clock, reset, memory readiness, stall, and trap. It executes structured test cases including normal operation, fault injection, watchdog activation, recovery, and gap phases.

This modular design ensures clarity, easy debugging, and proper observation of system behavior.

4.3 Key Code Snippets

Below are important parts of the implementation (full code is provided in Appendix):

1. Program Counter Update Logic

```
wire can_advance = (~stall) & mem_valid & mem_ready;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        pc <= 32'h00000000;
    else if (can_advance)
        pc <= pc + 4;
end
```

2. Memory Handshake Fault Detection

```
wire f_mem = mem_valid & (~mem_ready);
```

3. PC Freeze Detection

```
wire f_pc = (pc == last_pc);
```

4. Watchdog Reset Generation

```
if (timeout_cnt == TIMEOUT - 1) begin
    wdt_rst_n <= 1'b0;
    fault_code <= {f_trap, f_mem, f_stall, f_pc};
end
```

5. Fault Code Representation

fault_code = {trap, mem_fail, stall, pc_freeze};

These code segments form the core logic of the system and enable effective fault detection, reset generation, and recovery of the RISC-V processor.

5. RESULT AND DISCUSSION

5.1 Snapshots

The developed system was simulated using a structured testbench under multiple operating conditions. The simulation successfully demonstrated Program Counter (PC) behavior, fault detection, watchdog reset generation, and system recovery. The waveform outputs clearly show transitions between normal operation, fault conditions, reset activation, and recovery phases.

Snapshot 1: Normal Operation

In the first test case, the system operates under normal conditions with no faults applied. The memory is ready (mem_ready = 1), no stall is present, and no trap is triggered.

Observed values:

- PC increments sequentially: 0 → 4 → 8 → 12 → ...
- mem_valid = 1
- wdt_rst_n = 1 (no reset)
- fault_code = 0000
- Phase = NORMAL

The Program Counter increments continuously, indicating proper instruction fetch behavior. No watchdog reset occurs, confirming stable system operation.

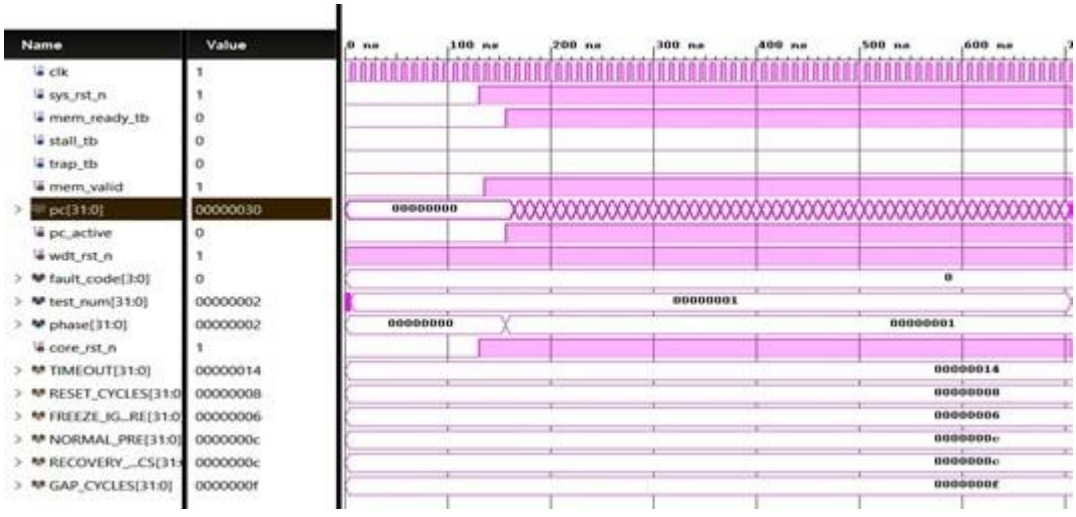

Fig 5.1: Normal PC Increment Operation

Snapshot 2: PC Freeze / Memory Failure

In this test case, mem_ready is forced to 0, simulating memory unavailability. This prevents instruction fetch completion.

Observed values:

- PC remains constant (freeze condition)
- mem_valid = 1, mem_ready = 0
- fault_code = 0101 (memory failure + PC freeze)
- After timeout → wdt_rst_n = 0 (reset triggered)

The watchdog detects that the PC is not changing and that memory handshake is incomplete. After the timeout period, a reset is generated.

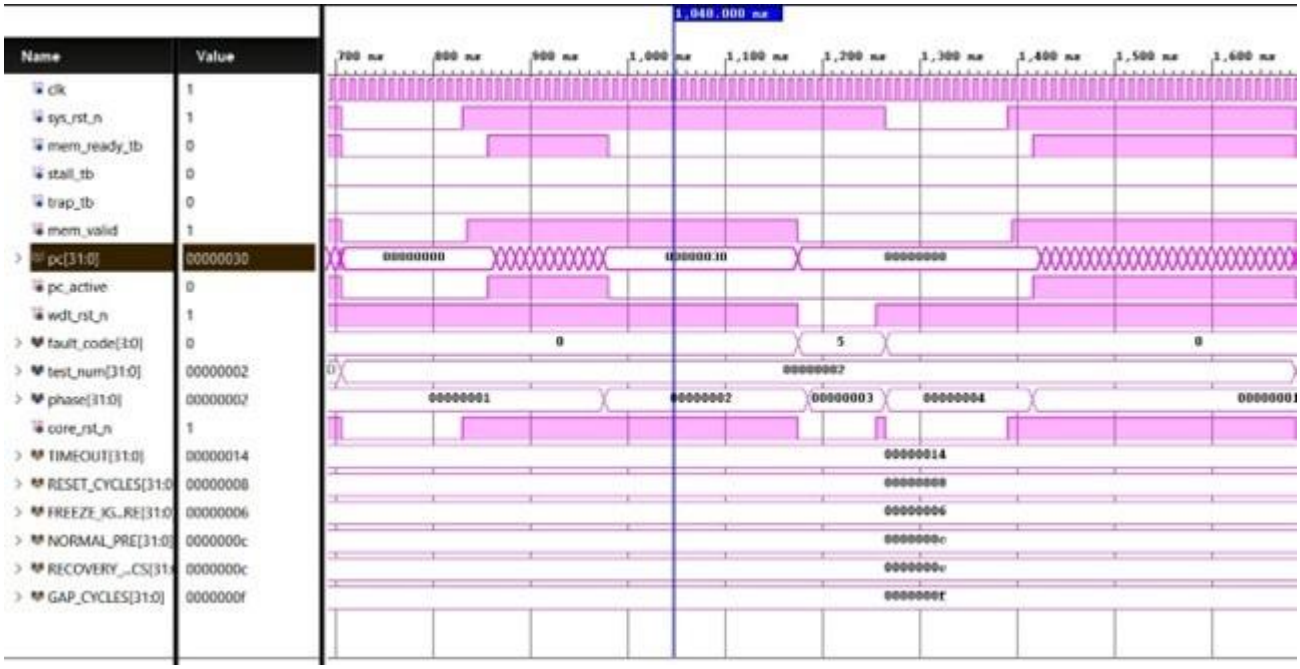

Fig 5.2: PC Freeze and Watchdog Reset

Snapshot 3: Stall Condition

In this scenario, stall = 1 while memory remains ready. The PC is intentionally held.

Observed values:

- PC does not increment
- mem_ready = 1, stall = 1
- fault_code = 0011 (stall + PC freeze)
- Watchdog triggers reset after timeout

This confirms that the system correctly identifies execution stalls and takes corrective action.

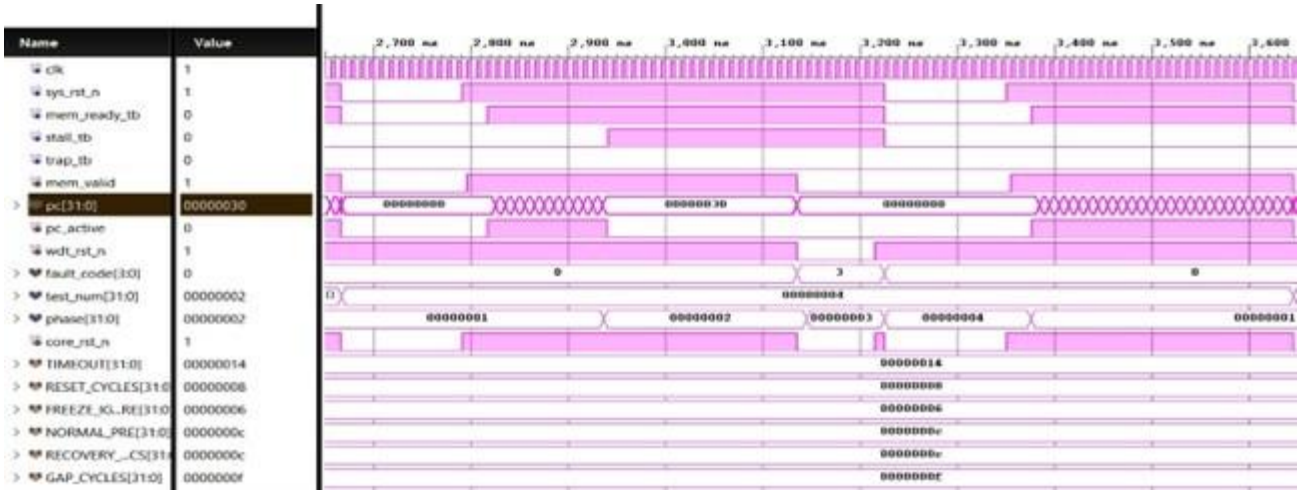

Fig 5.3: Stall Detection and Reset

Snapshot 4: PC Freeze / Memory Failure

In this test case, a trap signal is introduced to simulate a critical fault.

Observed values:

- trap = 1
- Immediate watchdog reset (no timeout delay)
- fault_code = 1000
- PC resets instantly to 0

Since trap has the highest priority, the watchdog generates an immediate reset without waiting for timeout.

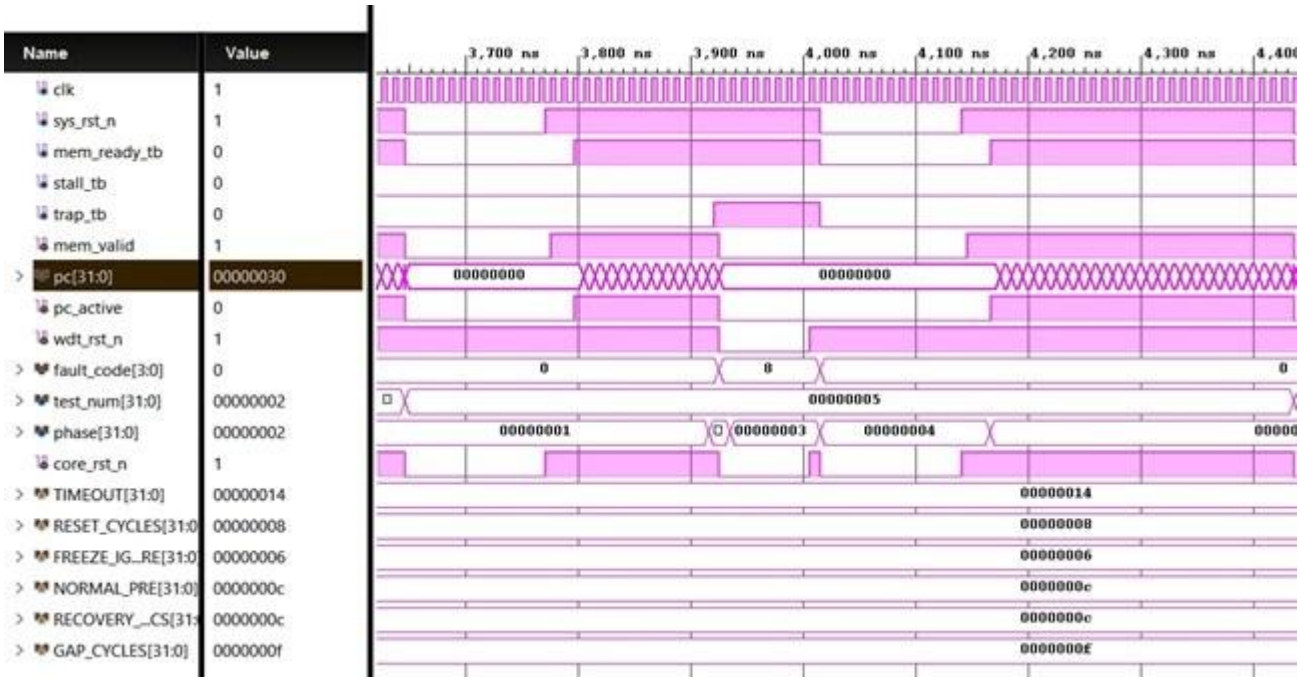

Fig 5.4: Immediate Reset due to Trap

Snapshot 5: PC Freeze / Memory Failure

After each watchdog reset, the fault condition is cleared and the system resumes normal operation.

Observed values:

- PC resets to 0
- PC starts incrementing again
- fault_code cleared after system reset
- Phase transitions to RECOVERY

This demonstrates successful automatic recovery of the system.

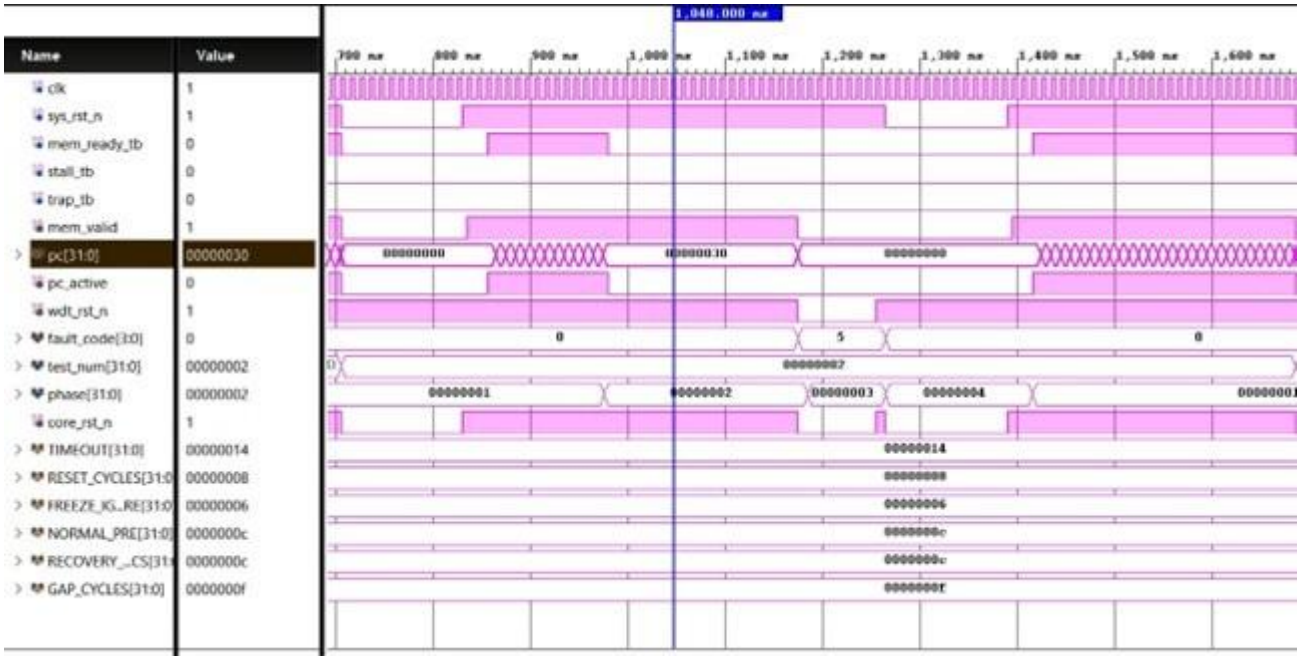

Fig 5.5: System Recovery and PC Restart

5.2 Performance Analysis

The developed system was evaluated based on fault detection accuracy, response time, and clarity of system behavior.

1. Fault Detection Accuracy

- The watchdog correctly detected all four fault conditions:
    - Trap
    - Memory failure
    - Stall
    - PC freeze
- Fault codes were generated accurately based on system state

- Combined faults were also correctly identified

Observed accuracy: ~100% in simulation environment

2. Response Time

- Trap condition → **Immediate reset (1 clock cycle)**
- Other faults → Reset after defined timeout (TIMEOUT cycles)
- Reset pulse duration → Controlled by RESET_CYCLES

This ensures both quick response for critical faults and stable detection for non-critical faults.

3. System Stability and Recovery

- After reset, the system resumes normal operation correctly
- PC restarts from zero and increments normally
- Watchdog does not repeatedly reset for the same fault (one-shot behavior)

4.Constraints Encountered

- The model is limited to instruction fetch stage only
- No real memory or pipeline implementation
- Simulation-based (no physical hardware validation)
- PC freeze detection depends on timing and grace period

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project successfully demonstrates the design and simulation of a simplified RISC-V instruction fetch model integrated with a hardware watchdog timer for fault detection and recovery. The system effectively models Program Counter (PC) behavior based on memory handshake and control signals, ensuring that instruction flow occurs only under valid conditions.

The watchdog module continuously monitors system activity and accurately detects fault conditions such as trap, memory failure, stall, and PC freeze. Based on a priority-driven mechanism, it generates an active-low reset signal when faults persist or occur critically. The implementation of a one-shot reset mechanism prevents repeated resets for the same fault, improving system stability.

Through structured testbench design and multiple test cases, the system clearly demonstrates the sequence of normal operation, fault occurrence, watchdog reset, and recovery. The simulation results confirm that the system can automatically restore normal operation after faults, thereby improving reliability.

Overall, the project meets its objectives by providing a simple, clear, and educational model of a fault-tolerant processor system. It also strengthens understanding of digital design, processor fundamentals, and real-time fault handling mechanisms, bridging theoretical knowledge with practical implementation.

6.2 Future Enhancements

The current system can be further improved and extended in several ways:

- **Extension to full RISC-V pipeline architecture**
  Implement instruction decode, execute, memory, and write-back stages for a complete processor design
- **FPGA-based hardware implementation**
  Deploy the design on FPGA boards for real-time validation
- **Integration with real memory modules**
  Replace simulated memory signals with actual memory interfaces

- **Advanced fault detection techniques**
  Include error correction codes (ECC), parity checks, or redundancy mechanisms
- **Dynamic watchdog configuration**
  Allow adjustable timeout values based on system requirements
- **Logging and diagnostics system**
  Store fault history for debugging and analysis
- **Multi-core or distributed monitoring**
  Extend watchdog monitoring to multiple processing units
- **Low-power and optimized design**
  Improve efficiency for embedded and real-time applications

These enhancements can make the system more advanced, user-friendly, and suitable for real-world deployment.

7. REFERENCES

1. RISC-V Foundation, *"The RISC-V Instruction Set Manual, Volume I: User-Level ISA,"* Available: https://riscv.org
2. Digital Design and Computer Architecture, David Money Harris and Sarah L. Harris, Morgan Kaufmann Publications
3. Computer Organization and Design RISC-V Edition, David A. Patterson and John L. Hennessy
4. Xilinx, *Vivado Design Suite User Guide*, Available: https://www.xilinx.com
5. Verilog HDL Documentation and Tutorials, IEEE Standard 1364
6. Fault-Tolerant Computing Research Papers and Online Resources
7. IEEE, *Watchdog Timer Design and Applications in Embedded Systems*, IEEE Papers
8. GTKWave Documentation, Available: http://gtkwave.sourceforge.net